

Author: Andrea Mussari

1,2)

The DP Table:

$T[i][j]$ = number of subsets using the first i items that achieve total weight exactly j .

$$T[i][j] = \begin{cases} 1 & \text{if } i = 0 \vee j = 0 \\ T[i-1][j] + T[i-1][j - w_{i-1}] & \text{otherwise} \end{cases}$$

The total number of feasible solutions is:

$$\sum_{j=0}^W T[n][j]$$

Uniform Sampling Procedure

- T computed
- $i = n$
- Choose initial weight j randomly from 0 to W , with probability proportional to $T[n][j]$
- $S = \emptyset$ = the selected subset ### Pseudocode

For ($i = n$) down to 1: If $j \geq w[i]$: $p_{\text{include}} = \frac{T[i-1][j-w[i]]}{T[i][j]}$; else : $p_{\text{include}} = 0$;
Draw randomly $r \in [0, 1)$; If $r < p_{\text{include}}$: include i -th item in S ; $j = j - w[i]$; $i--$;

Time Complexity:

Step	Complexity
DP Table construction	$O(nW)$
Compute total feasible count	$O(W)$
Sampling one solution	$O(n + W)$
Total (preprocessing + sampling)	$O(nW)$

3)

- $T[i][j]$ = maximum value achievable using a subset of the first i items with total weight $\leq j$

- $T[0][j] = 0$ for all j
- $T[i][0] = 0$ for all i
- $T[i][j] = \max(T[i-1][j], T[i-1][j-w_i] + v_i)$ if $j \geq w_i$
- $T[i][j] = T[i-1][j]$ otherwise
- $C[i][j]$ = number of optimal solutions for the subproblem (i,j)
- $C[0][j] = 1$ for all j
- $C[i][0] = 1$ for all i
- $\text{include_value} = T[i-1][j-w_i] + v_i$
- $\text{exclude_value} = T[i-1][j]$

$$C[i][j] = \begin{cases} C[i-1][j] & (j < w_i) \vee (\text{exclude_value} > \text{include_value}) \\ C[i-1][j-w_i] & \text{include_value} > \text{exclude_value} \\ C[i-1][j] + C[i-1][j-w_i] & \text{otherwise} \end{cases}$$

```
def sample_optimal_solution(T, C, weights, values, n, W):
    S = set() # Our selected items
    i = n
    j = W

    while i > 0:
        if j < weights[i-1]: # Item i doesn't fit
            i -= 1
        else:
            # Check if excluding or including is optimal
            exclude_value = T[i-1][j]
            include_value = T[i-1][j-weights[i-1]] + values[i-1]

            if exclude_value > include_value:
                # Must exclude
                i -= 1
            elif include_value > exclude_value:
                # Must include
                S.add(i)
                j -= weights[i-1]
                i -= 1
            else:
                # Both choices give optimal value
                exclude_count = C[i-1][j]
                include_count = C[i-1][j-weights[i-1]]
                total_count = exclude_count + include_count
```

```

        # Sample with probability proportional to counts
        r = random.random() * total_count
    if r < exclude_count:
        # Exclude item i
        i -= 1
    else:
        # Include item i
        S.add(i)
        j -= weights[i-1]
        i -= 1

return S

```